

Data Preparation

DAEN 300

Alexander Abuabara

Fall 2026



Where data pipeline fits in ML

- ▶ Data engineering builds the pipeline:
ingest → **clean** → **transform** → **store/serve**
- ▶ Machine learning consumes that clean data to answer a question: *is there a trend? can we predict a future value?*
- ▶ A data engineer rarely builds the fanciest model. They build the pipeline that makes a *correct, reproducible* model possible.
- ▶ Our example: FEMA disaster declaration records.

Loading and inspecting data

```
df = pd.read_csv('DisasterDeclarationsSummaries.csv')  
df.info()  
df = df.drop(columns=["lastRefresh", "hash", "id"])
```

- ▶ `read_csv` loads raw data into a DataFrame (70,405 rows, 29 columns).
- ▶ `df.info()` is the first data-engineering check: types, non-null counts, memory.
- ▶ `drop(columns=...)` removes housekeeping fields that carry no analytical value.

Data preparation: define the analysis

Goal: Analyze if the number of natural disasters/hazards are increasing over time.

1. Load the FEMA Disaster Declarations Summaries CSV.
2. Inspect column names, types, missing values, and example records.
3. Keep major disaster declarations (DR).
4. Exclude unnecessary incident categories.
5. Keep designated areas that are equivalents ... such as whose names contain county, parish, or municipio.
6. Check and exclude of issues in the data (e.g., fiscal year 2026, as it is still ongoing).

Check: Record the row count after each filter. A change in these rules changes the population being modeled.

Data preparation: filtering pseudocode

```
data := READ CSV("DisasterDeclarationsSummaries.csv")  
INSPECT schema, missing values, sample rows  
REMOVE unused metadata columns  
  
excluded := {Biological, Fishing Losses, Other,  
             Toxic Substances, Human Cause, Terrorist}  
  
clean := FILTER data WHERE  
    declarationType = DR  
    AND incidentType NOT IN excluded  
    AND designatedArea CONTAINS  
        county OR parish OR municipio  
    AND fyDeclared != 2026
```

Data preparation: dates and annual table

```
FOR EACH row IN clean:
    date := PARSE_DATE(row.declarationDate)
    row.year := CALENDAR_YEAR(date)
    row.month := CALENDAR_MONTH(date)
    row.year_month := FORMAT(date, "YYYY-MM")

annual := GROUP clean BY year
          COUNT DISTINCT disasterNumber AS disasterCount
SORT annual BY year

X := annual.year AS ONE-COLUMN TABLE
y := annual.disasterCount AS TARGET VECTOR
```

Check: One annual row represents one calendar year. Counting rows instead of distinct disaster numbers answers a different question. The exclusion uses fiscal year, while the derived year uses declaration date.

Filtering and grouping: query + groupby

```
df.query("declarationType == 'DR'")  
  .groupby(["state"])  
  .size()  
  .reset_index(name="count")  
  .sort_values("count", ascending=False)
```

- ▶ `query()` filters rows with a readable string expression (like SQL's `WHERE`).
- ▶ `groupby().size()` counts rows per category — the pandas equivalent of `GROUP BY ... COUNT(*)`.
- ▶ `reset_index()` turns the grouped result back into a flat, tidy DataFrame.
- ▶ This chain pattern (filter → group → aggregate → sort) is the core loop of exploratory data engineering.

Removing noise before modeling

```
excluded_incidents = ["Biological", "Fishing Losses",  
                      "Other", "Toxic Substances",  
                      "Human Cause", "Terrorist"]  
  
df.query("incidentType not in @excluded_incidents")
```

- ▶ @variable inside query() injects a Python list into the filter expression.
- ▶ Rare/irrelevant categories (e.g. COVID-19 as “Biological”) are excluded so they don’t distort a *weather-disaster* trend.
- ▶ **Data engineering lesson:** deciding what to exclude, and documenting why, is as important as any model choice.

Text cleanup: str accessor and explode

```
df["designatedArea"].str.lower().str.split().explode().value_counts()
```

- ▶ `.str.lower()` / `.str.split()` apply string operations element-wise on a column.
- ▶ `.explode()` turns each list entry into its own row — useful for tokenizing free text.
- ▶ `.value_counts()` tallies frequencies, here revealing that “county”, “parish”, “municipio” dominate — guiding the next filter.

Building engineered features with assign

```
df_clean = df.query(
    "declarationType == 'DR' "
    "and incidentType not in @excluded_incidents "
    "and designatedArea.str.contains(@included_areas, case=False)"
    "and fyDeclared != 2026",
    engine="python"
).assign(
    year_month=lambda x: pd.to_datetime(x["declarationDate"]).dt.
        strftime("%Y-%m"),
    year=lambda x: pd.to_datetime(x["declarationDate"]).dt.year,
    month=lambda x: pd.to_datetime(x["declarationDate"]).dt.month
)
```

- ▶ `assign()` creates new columns without mutating the original DataFrame (functional style).
- ▶ `pd.to_datetime(...).dt` extracts year/month — classic **feature engineering**: turning a raw date string into model-ready numeric fields.
- ▶ Excluding the incomplete current year (`fyDeclared != 2026`) avoids biasing the trend with partial data.

matplotlib: the plotting layer

```
import matplotlib.pyplot as plt

plt.scatter(annual["year"], annual["disasterDeclarations"], label="
    Observed")
plt.xlabel("Year")
plt.ylabel("Number of disasters")
plt.legend(loc="upper left")
plt.show()
```

- ▶ `plt.scatter` / `plt.plot` draw points or lines onto the current axes.
- ▶ `xlabel`, `ylabel`, `legend` annotate the figure — always label axes before sharing a plot.
- ▶ `plt.cla()` clears the current axes so successive plots don't overlap.
- ▶ Pandas' own `.plot()` method (also used in the script) is a thin convenience wrapper *around* matplotlib.

What “machine learning” means

- ▶ **Supervised learning:** learn a function $y = f(x)$ from labeled examples (x_i, y_i) .
- ▶ Here x = year (or date), y = number of disaster declarations — a **regression** problem (continuous target).
- ▶ **Training/fitting:** the model estimates parameters (slope, intercept) that best explain the observed data.
- ▶ **Prediction:** once fit, the model produces $\hat{y}$ for new or existing x values.
- ▶ **Evaluation:** how well do predictions match reality? (R^2 , residuals, error metrics.)

Linear regression, conceptually

$$\hat{y} = mx + b$$

- ▶ m (slope) and b (intercept) are learned so that the line minimizes squared error to the observed points.
- ▶ The script even computes m and b by hand with the closed-form formula, *before* introducing `sklearn` — showing that a library call is just an efficient implementation of this idea.
- ▶ R^2 measures the fraction of variance in y explained by x : $R^2 \in [0, 1]$, higher is better.

The sklearn estimator API

```
from sklearn.linear_model import LinearRegression

X = annual[["year"]] # features: must be 2D
y = annual["disasterDeclarations"] # target: 1D

model = LinearRegression()
model.fit(X, y) # learn parameters
model.coef_, model.intercept_ # learned slope/intercept
model.score(X, y) #  $R^2$  on training data
model.predict(X) # generate predictions
```

- ▶ Every sklearn model follows the same pattern: fit → predict/score.
- ▶ Features (X) are always 2D (rows = observations, columns = variables); target (y) is 1D.
- ▶ This consistent API is why sklearn scales from a one-line linear model to complex pipelines.

sklearn vs. statsmodels

- ▶ sklearn → optimized for **prediction**: fast fit/predict, easy to swap models, built for pipelines.
- ▶ statsmodels → optimized for **inference**: p-values, confidence intervals, full regression `summary()`.
- ▶ The script uses both on the *same* data: `sm.OLS(y, sm.add_constant(x)).fit()` for statistical rigor, `LinearRegression()` for a quick predictive fit.
- ▶ **Data engineering takeaway**: choose the tool based on the question — “how confident are we?” vs. “what’s the forecast?”

Does the time scale of the data matter?

- ▶ Same underlying data, aggregated two ways:
annual counts vs. **monthly** counts.
- ▶ Annual model:
slope $\approx 0.74/\text{year}$,
 $R^2 \approx 0.48$.
- ▶ Monthly model:
slope $\approx 0.00014/\text{month}$ ($\approx 0.60/\text{year}$ once rescaled),
 $R^2 \approx 0.09$.
- ▶ Lower monthly R^2 isn't a worse model! Monthly data has more noise (seasonality) that a simple line can't capture.
- ▶ The level of aggregation you choose upstream directly shapes what the model downstream can learn. **Data engineering implication!!**

Checking model assumptions: autocorrelation

```
from statsmodels.stats.stattools import durbin_watson  
  
residuals = y - model.predict(x)  
durbin_watson(residuals)
```

- ▶ Linear regression assumes independent residuals; time series data often violates this.
- ▶ Durbin–Watson statistic: ≈ 2 means little autocorrelation; < 2 means positive autocorrelation (nearby points are similar).
- ▶ Both annual ($DW \approx 1.33$) and monthly ($DW \approx 1.58$) models show some positive autocorrelation — a signal that a simple OLS trend is a simplification, useful but not the final word.

Summary

- ▶ **pandas** builds the pipeline: load, inspect, filter (query), aggregate (groupby), engineer features (assign).
- ▶ **matplotlib** makes the data (and the model's fit) visible before you trust it.
- ▶ **ML**, at its simplest, is fitting a function to data and checking how well it generalizes (R^2 , residual checks).
- ▶ **sklearn** standardizes the fit/predict/score workflow; **statsmodels** adds statistical inference.
- ▶ Clean, well-aggregated, well-labeled data is the real prerequisite for any of this to work!!